ArenaHub

Guia do backend

O que cada camada faz, por que ela existe
e por onde passa cada uma das 24 funcionalidades.

6	6	36	6	18	34
Models	Controllers	Services	Repositories	Procedures	Rotas

Flask + Flask-SQLAlchemy + SQLite · github.com/paulim21/hub_arena_ps

1. O caminho de uma requisicao

Toda funcionalidade do sistema faz o mesmo percurso. O exemplo abaixo e o de criar uma reserva, que e o mais completo: ele passa por todas as camadas e ainda consulta uma procedure no meio do caminho.

1. TELA `frontend/reservas.html`	Monta o JSON e chama fetch('/api/reservas', POST). Não calcula valor nenhum.
2. ROTA `backend/routes.py`	Mapa de URL para metodo de Controller. 34 rotas, nenhuma logica.
3. CONTROLLER `ReservaController.criar`	Le o corpo JSON, chama o Service, traduz excecao em status HTTP. Sem regra.
4. SERVICE `CriarReservaService.executar`	Valida, pergunta ao Repository se ha conflito, calcula horas x preco.
5. MODEL / REPOSITORY `Reserva.salvar / ReservaRepository`	Model persiste; Repository executa a procedure sp_conflito_reserva.
6. BANCO `backend/hub_arena.db`	SQLite com as 6 tabelas e a tabela procedures.

A pergunta que a banca faz: por que tantas camadas?

Porque cada uma responde por uma coisa so, e isso aparece quando algo muda. Trocar SQLite por MySQL mexe no Repository e nao toca no Service. Mudar a regra de cobranca mexe no Service e nao toca na Controller. Trocar a tela nao mexe em nada do backend, porque a tela so fala com a API. Em um arquivo unico, qualquer uma dessas mudancas obriga a reler tudo.

O valor da reserva e calculado no SERVIDOR, nunca na tela

O frontend manda quadra, inicio, fim e jogadores. Quem multiplica horas pelo preco da quadra e o CriarReservaService. Se o valor viesse da tela, bastaria abrir o console do navegador e mandar 'valor_total: 0'.

2. Mapa do backend, arquivo por arquivo

Arquivo ou pasta	Do que responde
`app.py`	Application Factory: cria a aplicacao, liga o SQLAlchemy, ativa as chaves estrangeiras do SQLite, cria as tabelas e registra as rotas.
`routes.py`	Mapa de URL -> metodo de Controller. Os Blueprints vivem aqui, e nenhuma logica mora neste arquivo.
`extensions.py`	O objeto db, criado sem vinculo com a app para que Models e Repositories possam importar sem import circular.
`database.py`	Registra as procedures no banco e aplica as migracoes de coluna (create_all nao altera tabela que ja existe).
`politica.py`	A versao e as finalidades dos termos de uso, num lugar so.
`validadores.py`	A forma de CPF, CNPJ, telefone e e-mail: digito verificador, DDD e normalizacao.
`models/`	6 classes que herdam de db.Model. Cada uma e uma tabela, e carrega o CRUD basico.
`controllers/`	6 classes, uma por recurso. Recebem HTTP e devolvem HTTP.
`services/`	36 classes, uma por caso de uso. Toda a regra de negocio.
`repositories/`	6 classes para consultas complexas, mais o runner que executa as procedures e o arquivo procedures.sql.

3. Models: as entidades do dominio

Herdam de `db.Model`, e e essa heranca que mapeia cada classe a uma tabela. O CRUD basico fica na propria Model, como pede a arquitetura da disciplina: `salvar()`, `atualizar()`, `deletar()`, `listar_todos()` e `buscar_por_id()`.

Model	Tabela	Colunas	O que representa
Arena	`arenas`	8	Pertence a um dono (FK para usuarios) e agrega quadras.
Consentimento	`consentimentos`	8	O aceite dos termos, com versao, data, IP e navegador. Nao tem FK para Usuario de proposito (ver secao 8).
Quadra	`quadras`	5	Pertence a uma arena. O preco por hora daqui e a base do valor da reserva.
Reserva	`reservas`	7	Quadra, periodo, valor e status. salvar() e redefinida para gravar a reserva e os jogadores na mesma transacao.
ReservaJogador	`reserva_jogadores`	2	Classe associativa que resolve o N:N entre Reserva e Usuario. Chave primaria composta, e ON DELETE CASCADE.
Usuario	`usuarios`	8	Atleta, dono ou professor. E-mail e CPF sao UNIQUE; a senha e guardada como resumo scrypt, nunca em texto.

Relacionamentos, na linguagem do diagrama de classes

Composicao (a parte nao existe sem o todo): Arena para Quadra, e Reserva para ReservaJogador. **Agregacao** (a parte existe por si so): Usuario para Arena, Quadra para Reserva, Usuario para Reserva e Usuario para ReservaJogador. O diagrama completo esta em docs/diagrama-classes.svg.

4. Controllers: a fronteira com o HTTP

Sao classes, uma por recurso. Fazem tres coisas e nada mais: leem o que veio na requisicao, chamam o Service correspondente e traduzem o resultado (ou a excecao) em resposta HTTP com o status certo.

Controller	Metodos	Recurso
ArenaController	6	`/api/arenas`
ConsentimentoController	5	`/api/consentimentos`
QuadraController	6	`/api/quadras`
RelatorioController	5	`/api/relatorios`
ReservaController	5	`/api/reservas`
UsuarioController	6	`/api/usuarios`

A traducao de excecao em status e a convencao do projeto:

Excecao no Service	Status HTTP	Quando acontece
ValueError	400	Dado invalido ou regra de negocio violada (CPF errado, horario em conflito)
LookupError	404	O registro pedido nao existe
IntegrityError	400	O banco recusou (UNIQUE ou chave estrangeira), e o Service traduz em mensagem legivel

O que NAO pode aparecer numa Controller

Calculo, validacao de regra, SQL e chamada de procedure. Se a banca abrir um arquivo de controller e encontrar um 'if' decidindo preco, a separacao de camadas caiu.

5. Services: um por caso de uso

Cada classe responde por **uma acao** do sistema e expoe um unico metodo `executar()`. E aqui que mora toda a regra de negocio, e e por isso que sao 36 classes e nao seis.

```
AgendaArenaService                CriarQuadraService                ListarUsuariosService
AtualizarArenaService             CriarReservaService               OcupacaoQuadrasService
AtualizarQuadraService            CriarUsuarioService               PesquisarUsuariosService
AtualizarStatusReservaService     DeletarArenaService               QuadrasDisponiveisService
AtualizarUsuarioService           DeletarQuadraService              RankingAtletasService
BuscarArenaService                DeletarReservaService             ReceitaPorArenaService
BuscarQuadraService               DeletarUsuarioService             RegistrarConsentimentoService
BuscarReservaService              ListarArenasComDonoService        RelatorioConsentimentosService
BuscarUsuarioService              ListarArenasService               ReservasRecentesService
ConferirSenhaService              ListarQuadrasService              ResumoDashboardService
ConsultarPoliticaService          ListarReservasDetalhadasService   RevogarConsentimentoService
CriarArenaService                 ListarReservasService             VerificarConsentimentoService
```

Os tres Services que vale abrir na apresentacao

Service	Por que ele e interessante
CriarReservaService	Concentra as duas regras mais citadas do projeto: pergunta ao Repository se ha conflito de horario (anti-overbooking) e calcula o valor como horas x preco da quadra. Tambem aceita varios jogadores, gravando o N:N.
CriarUsuarioService	Valida a forma dos dados antes de tocar o banco (CPF com digito verificador, e-mail normalizado, telefone com DDD) e transforma a senha em resumo scrypt. Quando o UNIQUE estoura, diz QUAL campo colidiu.
RegistrarConsentimentoService	Recusa consentimento parcial e versao desatualizada dos termos, e revoga o aceite anterior do mesmo aparelho antes de gravar o novo.

6. Repository e as procedures

O Repository nao repete CRUD simples: isso ja esta na Model. Ele existe para **consulta complexa** — JOIN, GROUP BY, NOT EXISTS, ranking, relatorio — e e o unico lugar do sistema que executa SQL.

Como existem procedures em SQLite

O SQLite e um banco embarcado e nao tem CREATE PROCEDURE, que e recurso de SGBD cliente-servidor. Para atender ao requisito, cada procedure e **armazenada na propria tabela procedures do banco** (nome + SQL parametrizado), registrada na subida do servidor a partir de repositories/procedures.sql. Quem faz o papel do CALL e o procedure_runner.py, dentro do Repository. Controller e Service nunca executam SQL.

As 18 procedures

Procedure	Para que serve
`sp_usuarios_por_tipo`	Filtro de usuarios por perfil
`sp_busca_usuarios`	Busca textual por nome ou e-mail (LIKE)
`sp_arenas_por_cidade`	Filtro de arenas por cidade
`sp_arenas_com_dono`	JOIN com o dono + contagem de quadras
`sp_quadras_com_arena`	JOIN quadra/arena
`sp_quadras_por_arena`	Quadras de uma arena
`sp_conflito_reserva`	ANTI-OVERBOOKING: reservas que colidem no periodo
`sp_reservas_detalhadas`	JOIN triplo reserva/quadra/arena
`sp_reservas_recentes`	Ultimas reservas do painel
`sp_jogadores_da_reserva`	Nomes dos jogadores (JOIN no N:N)
`sp_resumo_dashboard`	Contadores e receita total
`sp_receita_por_arena`	RELATORIO: JOIN + GROUP BY + ticket medio
`sp_ocupacao_quadras`	RELATORIO: horas reservadas e receita por quadra
`sp_ranking_atletas`	RELATORIO: ranking por jogos e horas
`sp_quadras_disponiveis`	BUSCA: quadras livres no periodo (NOT EXISTS)
`sp_agenda_arena`	Agenda da arena no periodo
`sp_consentimento_vigente`	LGPD: o aceite ainda valido deste aparelho
`sp_consentimentos_por_versao`	LGPD: aceites, vigentes e revogados por versao

7. As 24 funcionalidades, de ponta a ponta

Cada linha e um caminho completo: tela, rota, Controller, Service e, quando ha consulta complexa, a procedure que o Repository executa.

#	Funcionalidade	Rota	Service	Procedure
1	Cadastrar usuario	`POST /api/usuarios`	`CriarUsuarioService`	-
2	Listar/filtrar por tipo	`GET /api/usuarios?tipo=`	`ListarUsuariosService`	`sp_usuarios_por_tipo`
3	Buscar por nome ou e-mail	`GET /api/usuarios/busca`	`PesquisarUsuariosService`	`sp_busca_usuarios`
4	Excluir usuario	`DELETE /api/usuarios/`	`DeletarUsuarioService`	-
5	Cadastrar arena	`POST /api/arenas`	`CriarArenaService`	-
6	Listar arenas com dono	`GET /api/arenas?detalhes=1`	`ListarArenasComDonoService`	`sp_arenas_com_dono`
7	Excluir arena	`DELETE /api/arenas/`	`DeletarArenaService`	-
8	Cadastrar quadra	`POST /api/quadras`	`CriarQuadraService`	-
9	Listar/filtrar quadras	`GET /api/quadras`	`ListarQuadrasService`	`sp_quadras_com_arena`
10	Excluir quadra	`DELETE /api/quadras/`	`DeletarQuadraService`	-
11	Criar reserva	`POST /api/reservas`	`CriarReservaService`	`sp_conflito_reserva`
12	Listar reservas detalhadas	`GET /api/reservas?detalhes=1`	`ListarReservasDetalhadasService`	`sp_reservas_detalhadas`
13	Confirmar reserva	`PUT /api/reservas/`	`AtualizarStatusReservaService`	-
14	Cancelar reserva	`DELETE /api/reservas/`	`DeletarReservaService`	-
15	Painel com resumo	`GET /api/relatorios/resumo`	`ResumoDashboardService`	`sp_resumo_dashboard`
16	Receita por arena	`GET /api/relatorios/receita-arenas`	`ReceitaPorArenaService`	`sp_receita_por_arena`
17	Ocupacao das quadras	`GET /api/relatorios/ocupacao-quadras`	`OcupacaoQuadrasService`	`sp_ocupacao_quadras`
18	Ranking de atletas	`GET /api/relatorios/ranking-atletas`	`RankingAtletasService`	`sp_ranking_atletas`
19	Quadras disponiveis	`GET /api/quadras/disponiveis`	`QuadrasDisponiveisService`	`sp_quadras_disponiveis`
20	Agenda da arena	`GET /api/arenas//agenda`	`AgendaArenaService`	`sp_agenda_arena`
21	Registrar consentimento	`POST /api/consentimentos`	`RegistrarConsentimentoService`	-
22	Verificar consentimento	`GET /api/consentimentos/vigente`	`VerificarConsentimentoService`	`sp_consentimento_vigente`
23	Revogar consentimento	`POST /api/consentimentos/revogar`	`RevogarConsentimentoService`	-
24	Relatorio de consentimentos	`GET /api/consentimentos/relatorio`	`RelatorioConsentimentosService`	`sp_consentimentos_por_versao`

8. O consentimento LGPD no backend

O sistema nao abre antes do aceite: o frontend segura toda chamada a API ate existir consentimento. Mas a parte que importa para a defesa esta no backend.

O que a lei pede	Onde isso esta no codigo
Comprovar o consentimento obtido (Art. 8, par. 1)	Cada aceite e uma linha na tabela consentimentos, com versao dos termos, data, IP e navegador. Um sinalizador no navegador nao comprova nada.
Consentimento por finalidade; o generico e nulo (Art. 8, par. 4)	politica.py declara as finalidades separadas, e o RegistrarConsentimentoService recusa aceite parcial.
Novo aceite quando a finalidade muda (Art. 8, par. 6)	O campo versao_termos. O Service recusa aceite de versao que nao e a vigente; subir a VERSAO faz todo mundo aceitar de novo.
Revogacao livre e gratuita (Art. 8, par. 5)	RevogarConsentimentoService preenche revogado_em. O registro NAO e apagado: ele e a prova, inclusive da revogacao.
Medidas de seguranca (Art. 46)	A senha vai ao banco como resumo scrypt com sal. Nem a equipe consegue ler a senha de um usuario.
Minimizacao (Art. 6, III)	Consentimento guarda um identificador ALEATORIO do navegador, e nao o usuario_id. Por isso a Model nao tem chave estrangeira para Usuario.

9. Validacao e seguranca

O arquivo `validadores.py` conhece a forma dos dados que tem forma. Ele fica fora dos Services de proposito: nao e caso de uso, e a forma do dado, e ela e a mesma para quem cadastra e para quem atualiza.

Dado	O que e conferido	Como vai ao banco
CPF	11 digitos, digitos verificadores calculados, e recusa de sequencias repetidas como 111.111.111-11	12345678909
CNPJ	14 digitos e digitos verificadores (opcional na arena)	11222333000181
Telefone	DDD valido e 8 ou 9 digitos; celular de 9 precisa comecar com 9; o 55 do pais e descartado	31999990000
E-mail	formato, e normalizacao para minusculas (senao Ana@x.com e ana@x.com viram duas contas)	ana@email.com
Senha	minimo de 6 caracteres	resumo scrypt com sal

A pontuacao e apresentacao, e apresentacao nao se persiste

O banco guarda 12345678909, nunca 123.456.789-09. Gravar formatado obriga todo filtro, toda busca e todo indice a adivinhar qual das grafias veio. A mascara aparece ao digitar, no frontend, e a formatacao para leitura acontece na tela.

Integridade no banco

O SQLite so respeita chave estrangeira se o PRAGMA for ligado **em cada conexao**, e e isso que o app.py faz. E por causa disso que excluir um dono com arena, uma arena com quadras ou uma quadra com reservas e recusado — e o Service traduz esse erro do banco em uma frase que a pessoa entende.

10. Como rodar e o que mostrar

Subir o projeto

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python backend/app.py
```

O banco `backend/hub_arena.db` e criado sozinho na primeira execucao, com as tabelas e as procedures registradas. Painel em localhost:5000; a API responde em /api/... e pode ser aberta direto no navegador ou no Postman.

A sequencia que mostra o backend inteiro em quatro minutos

Passo	O que dizer enquanto mostra
Aceitar os termos	O aviso aparece antes de tudo e os paineis atras estao vazios: nenhuma chamada a API acontece sem consentimento, e o aceite vai para o banco.
Cadastrar um usuario	Digitar o CPF devagar para a mascara aparecer, e citar que o servidor confere os digitos verificadores e guarda so os numeros.
Criar uma reserva	Apontar o valor na tabela: ele veio do servidor. Em seguida tentar o mesmo horario na mesma quadra e mostrar o bloqueio.
Abrir os relatorios	Receita por arena (JOIN + GROUP BY), ocupacao, ranking e quadras disponiveis (NOT EXISTS). Todos sao procedure chamada pelo Repository.
Abrir o codigo	reserva_controller.py (so recebe e devolve), reserva_service.py (a regra), reserva.py (o CRUD na Model) e procedures.sql (o SQL).

11. Perguntas provaveis, e o que responder

Por que o Repository nao tem os metodos de CRUD?

Porque o CRUD simples ja esta na Model, como a arquitetura da disciplina pede. Criar um Repository so para repetir save e delete seria uma camada sem trabalho. O Repository existe para o que a Model nao faz: JOIN, GROUP BY, NOT EXISTS e as procedures.

Isso e procedure de verdade? SQLite nao tem CREATE PROCEDURE.

Nao tem mesmo, e por isso o projeto guarda cada procedure na tabela procedures do proprio banco, com o SQL parametrizado, e as executa pelo procedure_runner. O que o requisito pede — SQL fora do codigo, encapsulado no Repository, chamado por nome — esta cumprido. Controller e Service nunca executam SQL.

Por que 36 Services? Nao daria para ter um por recurso?

Daria, mas ai cada classe teria varias responsabilidades. O criterio adotado e um Service por CASO DE USO, entao cadastrar usuario e listar usuarios sao classes diferentes, com um metodo executar() cada.

Onde esta a regra de negocio?

Sempre no Service. A Controller so traduz HTTP, a Model so persiste e o Repository so consulta. O anti-overbooking e o calculo do valor estao no CriarReservaService.

Como o sistema impede duas reservas no mesmo horario?

O Service chama ReservaRepository.existe_conflito, que executa a procedure sp_conflito_reserva: ela procura reserva na mesma quadra cujo inicio seja menor que o fim pedido e cujo fim seja maior que o inicio pedido. Se achar, o Service levanta ValueError e a Controller devolve 400.

E se o usuario mandar o valor da reserva pela API?

E ignorado. O CriarReservaService nem le esse campo: ele calcula horas vezes o preco da quadra que esta no banco.